Task 6: Benchmark Your Design

Group 11

1 Introduction

This benchmark evaluates the scalability of the Taxi Trips storage design. Two partitioning strategies were implemented for the `integrated_taxi_trips` Gold Delta table. We compare ingestion time, storage size, query latency, and number of generated files.

1.1 Reproducibility

The benchmark implementation is provided in `notebooks/benchmark.ipynb`. The notebook runs the ingestion pipeline for both storage designs, collects storage and file metrics, and measures query latency.

The notebook exports complete query outputs to `notebooks/benchmark/`. To reproduce the benchmark, start the project services, check and follow the `Readme.md` at `Data-Ingestion-Framework/notebooks/Readme.md` and run the notebook cells from top to bottom.

2 Storage Strategies

Both strategies execute the same complete pipeline:

`landing` → `bronze` → `silver` → `gold`

The only difference is the partition column used when writing the final Gold `integrated_taxi_trips` Delta table.

Strategy	**Gold partition column**	**Expected trade-off**
Date partitioning	`date`	Can be faster for queries that select a specific day or date range. However, it creates one partition for every date, which can produce more files.
Borough partitioning	`pickup_borough`	Can be faster for queries by borough. It creates fewer partitions and usually fewer files, but is less useful for date-based queries.

Table 1: Storage strategies evaluated.

3 Methodology

3.1 Benchmark scope

For each strategy, the timer starts before the pipeline reads the landing datasets and stops after the Gold Delta table has been written. Therefore, the ingestion-time measurement includes Bronze ingestion, Silver transformation and validation, Gold integration, repartitioning, and the final Delta write.

Storage size and generated file counts are measured only for the final Gold table being compared. This avoids including shared Bronze, Silver, lookup, weather, and air-quality storage costs that are not affected by the selected Gold partitioning strategy.

3.2 Metrics

- **Ingestion time:** end-to-end elapsed time in seconds.
- **Data size:** total size of Parquet data files in MiB.
- **Total size:** Parquet data plus Delta transaction-log files.
- **Generated files:** number of Parquet data files.
- **Total files:** Parquet data files plus Delta-log files.
- **Query latency:** one warm-up execution followed by three timed executions. The median is reported.

4 Results

4.1 Ingestion and Storage

Strategy	Partition column	Ingestion time (s)	Data size (MiB)	Total size (MiB)	Data files	Total files
Date partitioning	`date`	93.83	281.78	282.23	165	167
Borough partitioning	`pickup_borough`	113.29	327.96	328.14	32	34

Table 2: End-to-end ingestion and final Gold-table storage measurements.

4.2 Query Latency

Query	Storage strategy	Median latency (s)	Timed samples (s)
Trips per borough	Date partitioning	0.366	(0.405, 0.366, 0.331)
Trips per borough	Borough partitioning	0.168	(0.168, 0.149, 0.171)
Average trip duration per day	Date partitioning	0.393	(0.411, 0.393, 0.381)
Average trip duration per day	Borough partitioning	0.250	(0.244, 0.250, 0.258)
Average fare per borough	Date partitioning	0.341	(0.346, 0.300, 0.341)
Average fare per borough	Borough partitioning	0.172	(0.172, 0.185, 0.163)

Table 3: Query-latency results. Each reported value is the median of three timed executions after one warm-up execution.

4.3 Query Outputs

Complete query outputs for both storage strategies are available in the repository under `notebooks/benchmark/`.

5 Discussion

Ingestion performance. Date partitioning completed the end-to-end pipeline in 93.83 seconds, while borough partitioning required 113.29 seconds. Borough partitioning was therefore approximately 20.7% slower. Although it creates fewer output files, grouping the data by borough does not automatically make the write faster.

Storage and generated files. The date-partitioned table produced 165 data files, compared with only 32 files for borough partitioning. This is expected because the dataset contains many more distinct dates than pickup boroughs. However, the date-partitioned table was smaller: 281.78 MiB of data compared with 327.96 MiB for borough partitioning. This shows that fewer partition directories and files do not necessarily result in a smaller Delta table.

Query performance. Borough partitioning was the fastest strategy for all three queries. It creates 32 data files, compared with 165 files for date partitioning. Reading fewer files likely reduces Spark's overhead.

The required queries calculate results for the complete dataset. They do not filter the data with conditions such as `WHERE date = ...` or `WHERE pickup_borough = ...`. Therefore, Spark cannot skip many partitions through partition pruning. The lower query latency of borough partitioning is more likely caused by its smaller number of larger data files than by Spark reading less taxi-trip

data. This is consistent with best practices documentation from Delta Lake Best Practices, because even though we query by date, and thus expect the date partitioning scheme to be faster, since each date partition is small, the query is slower compared to the Borough partitioning scheme.

Scalability. Both strategies completed the pipeline in under two minutes and returned equivalent results. Date partitioning is better for lower ingestion time and storage size. Borough partitioning creates fewer files and was faster for all tested queries, but required more ingestion time and storage.

6 Conclusion

The benchmark shows that borough partitioning is the more suitable strategy for the current analytical workload. Although date partitioning had a faster end-to-end ingestion time and a smaller final Gold table, borough partitioning created far fewer files and was faster for all three required queries. Therefore, the choice is based on the trade-off between ingestion and storage efficiency versus query latency.